

01 / INSTRUCTOR GUIDE

Payment Rails Live

Watch a card payment travel through six programs

USF AI in FinTech | Project 0 | Instructor demonstration

Why start here?

A payment approval is a decision. It does not mean the merchant has already received the sale proceeds. This demonstration makes the decisions, messages, temporary holds, and later financial effects visible, one message at a time.

Six small Python processes represent the payer, merchant/POS, gateway/processor, acquirer, simulated Visa or Mastercard network, and issuer. An animated browser dashboard follows actual messages from those processes. The programs use deterministic rules; no language model chooses whether a payment succeeds.

The classroom question

A payer has \$500. A merchant requests \$50. When the POS says Approved, where is the money? Show the hold, capture the payment, clear the batch, then settle it. Explain why the merchant receives \$48.75.

What the audience will observe

- The request travels forward; the issuer's decision travels back through every participant.
- Authorization reserves \$50: posted balance remains \$500; available balance becomes \$450.
- Capture records the merchant's intent to collect. Clearing posts a \$50 debit in this toy model.
- Settlement credits \$48.75 to the merchant and \$1.25 to illustrative fee allocations.
- A decline, a duplicate, and an invalid transition produce visible reasons without inventing money movement.

Use the delivered implementation

Start the project, project the dashboard, and run the script on pages 7–8. The package contains working source, fictional fixtures, automated tests, editable diagrams, two PDFs with LaTeX sources, screenshots, and a verification record. This is an unscored instructor example, with an observation checklist rather than a submission requirement.

Teaching boundaries

One USD debit-card flow, one instructor session, full capture, temporary memory, and accelerated batches keep the implementation small. No real cards, bank connections, model API, refunds, chargebacks, partial capture, or crash recovery are included. Debit posting at clearing is a teaching convention; real scheme and issuer timing varies.

02 / INSTRUCTOR GUIDE

Six actors, six responsibilities

The six boxes are distinct running programs. In real commerce, a single provider can bundle several roles. The separation here makes the responsibilities easier to see.

Actor	What its program demonstrates
Payer	Starts a fictional purchase and receives its final outcome. The issuer owns the authoritative account balance.
Merchant / POS	Accepts the purchase, sends the authorization request, records the returned decision, and initiates full capture.
Gateway / processor	Checks the request contract, preserves identifiers, and routes valid payment operations.
Acquirer	Represents the merchant side, tracks authorized/cleared records, and records merchant proceeds plus illustrative fee allocations.
Simulated card network	Routes Visa or Mastercard messages, checks batch membership and totals, and validates batch membership and totals.
Issuer	Looks up the synthetic account, checks account status and available funds, owns holds and posted balances, and processes clearing/settlement obligations.

Three words to distinguish

Word	Meaning in this demonstration
Message	A request or response that one actor sends to another. Message animation represents communication.
State	What an actor currently remembers: an authorization, hold, capture, balance, batch, or fee allocation.
Agent	A small program with a role, local state, and rules for handling incoming messages. It does not require AI or an LLM.

Narration to use at the diagram

“The merchant asks for permission. The issuer makes the account decision. The network carries that decision; the acquirer connects the merchant side. Later, the approved sale becomes a claim for payment and is settled.”

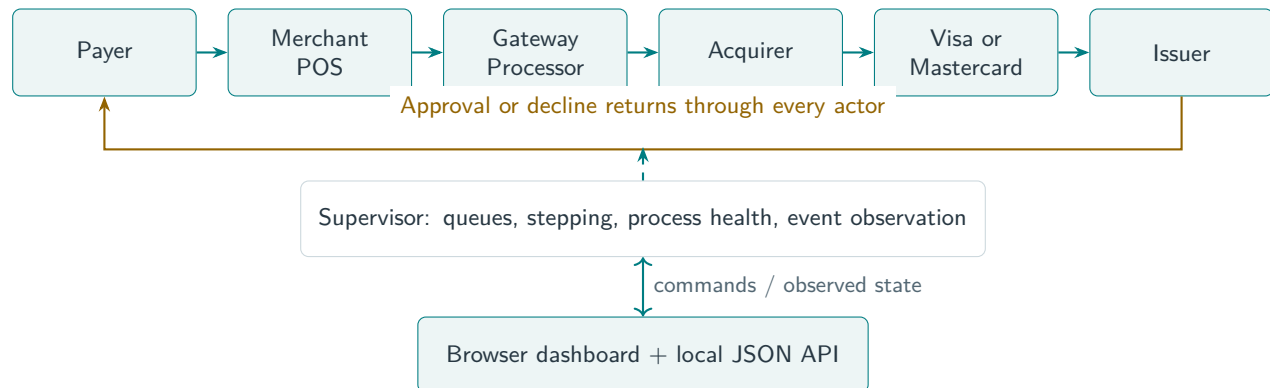
A gateway is especially visible in online payments. A physical POS can connect through different processor arrangements. Project 0 deliberately uses the same six-hop route for every purchase so the audience can follow one consistent example.

Evidence to point to

Each card on the dashboard shows its process ID and latest activity. A message record shows sender, recipient, payment ID, operation ID, amount, and outcome. The PID is evidence that a role is running separately; the trace is evidence of what it actually handled.

03 / INSTRUCTOR GUIDE

Architecture you can point to



What starts with one command

The parent Python program starts six child processes, then serves bundled HTML, CSS, and JavaScript at <http://127.0.0.1:8010>. Each role has an inbox queue. The supervisor delivers the next queued message and collects the resulting events and state snapshots.

The supervisor controls pacing and displays observations. It does not approve transactions or manufacture balances. Those rules and balances belong to participant programs.

Read the diagram in two directions

The top path carries an authorization request toward the issuer. The lower return path carries the issuer's result back to the payer. Capture is a local merchant operation. Clearing and settlement travel from merchant through gateway, acquirer, and network to issuer, then return to merchant.

Why this is a useful minimal implementation

- Local queues avoid account setup, network services, and paid APIs.
- Separate processes show the boundary between transport and participant decisions.
- A shared event stream connects console evidence to the browser animation.
- Source diagrams in docs/diagrams can be edited as Mermaid; SVG exports can be inserted into slides.

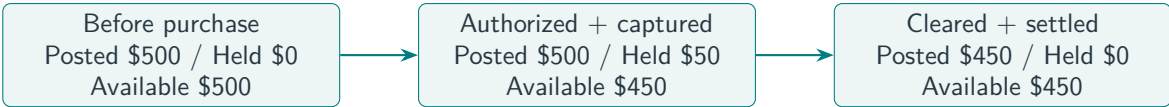
04 / INSTRUCTOR GUIDE

Follow one \$50 purchase

Start from Reset. Use the approved preset and remain in Guided mode. Advance messages until the transaction reaches the stage named below; a button click can enqueue work before the final response returns.

Stage	Instructor action	What the audience should see
Authorization	Create purchase; Next Message repeatedly	Forward request, issuer checks, reverse approval. Posted \$500; hold \$50; available \$450.
Capture	Select payment; Capture; advance	Merchant confirms full \$50 collection. The hold remains. No merchant proceeds yet.
Clearing	Clear Batch; advance	Captured sale is validated; issuer releases the hold and posts \$50. Posted \$450; held \$0; obligation \$50.
Settlement	Settle Batch; advance	Cleared batch membership/amounts reconcile. Obligation becomes \$0; merchant receives \$48.75.

The visible account arithmetic



Available funds equal posted balance minus active holds. After clearing, available funds stay \$450: a hold has been replaced by a posted debit, not charged a second time.

Where \$1.25 goes in the toy fee policy

Allocation	Policy	\$50 example
Issuer interchange	2%	\$1.00
Network fee	0.1%	\$0.05
Processor/acquirer fee	0.2% + \$0.10	\$0.20
Merchant proceeds	Gross less all fees	\$48.75

These rates are invented for arithmetic clarity. They are not Visa, Mastercard, Stripe, or any bank’s price list. Integer-cent components are rounded separately, half up. The toy rejects a purchase if its configured fees exceed its amount.

05 / INSTRUCTOR GUIDE

Start the local demonstration

Requirements

Use Python 3.9 or later and a current browser. The runtime uses Python's standard library and bundled assets. No pip install, cloud account, card credentials, or model API key is needed. Purchases use integer cents from 100 through 1000000 (\$1–\$10,000).

Unzip the full package and keep its directory structure. Open a terminal in `project-0-payment-rails`. Run:

```
python3 start.py
```

On Windows PowerShell use:

```
py -3 start.py
```

The delivered launchers provide an alternative to typing the command. The README names them and lists options. A terminal remains visible with role IDs and readable message logs. Keep that terminal open throughout the demonstration.

Before students enter the room

1. Open <http://127.0.0.1:8010>. Confirm six participant cards report running process IDs.
2. Choose Guided mode and reset. Start the approved preset and advance a few messages.
3. Check that the highlight follows the trace. Reset again for the class opening.
4. Set browser zoom for the projector. Use a wide screen so all six actors remain easy to follow.

Stop and restart

Press Ctrl+C in the launch terminal. The supervisor shuts down the server and child processes. Relaunching starts fresh fictional accounts and history. Reset also restarts all participants together; browser refresh alone is not a financial reset.

If port 8010 is already occupied

Stop your prior demonstration or choose another local port:

```
python3 start.py --port 8011 --no-browser
```

Open <http://127.0.0.1:8011>. The browser uses relative API paths, so it follows the chosen port.

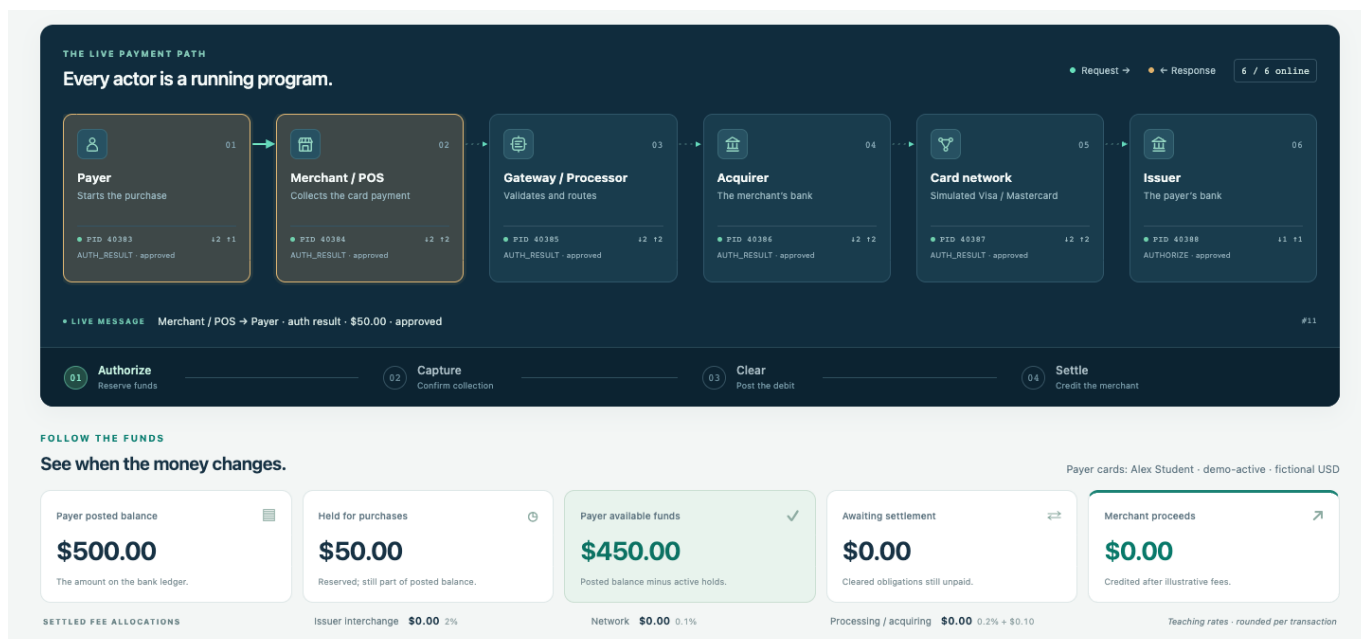
Container and cloud path

Python is the lowest-friction classroom route. The Dockerfile packages the same application. The technical supplement gives local container commands and VM/SSH-tunnel paths for AWS, GCP, and Azure. Those are deployment instructions, not claims that cloud resources have been provisioned.

06 / INSTRUCTOR GUIDE

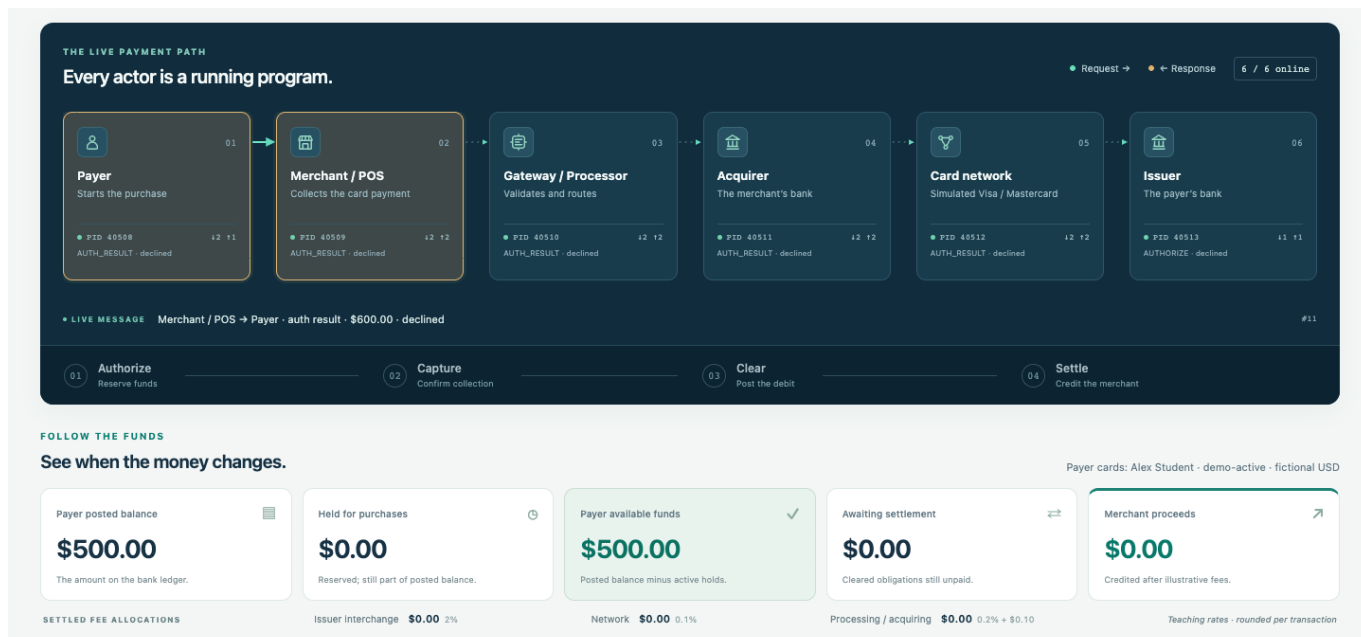
Screens from the running dashboard

A. Approved authorization. These are detail crops of the delivered running application. Full captures are included in the screenshots folder; PIDs and timestamps vary.



Explain the forward request and reverse response. Point to the issuer's hold, the payer's available balance, and the selected message's payment ID.

B. Declined authorization. Reset, choose insufficient funds, and advance to the returned decline.



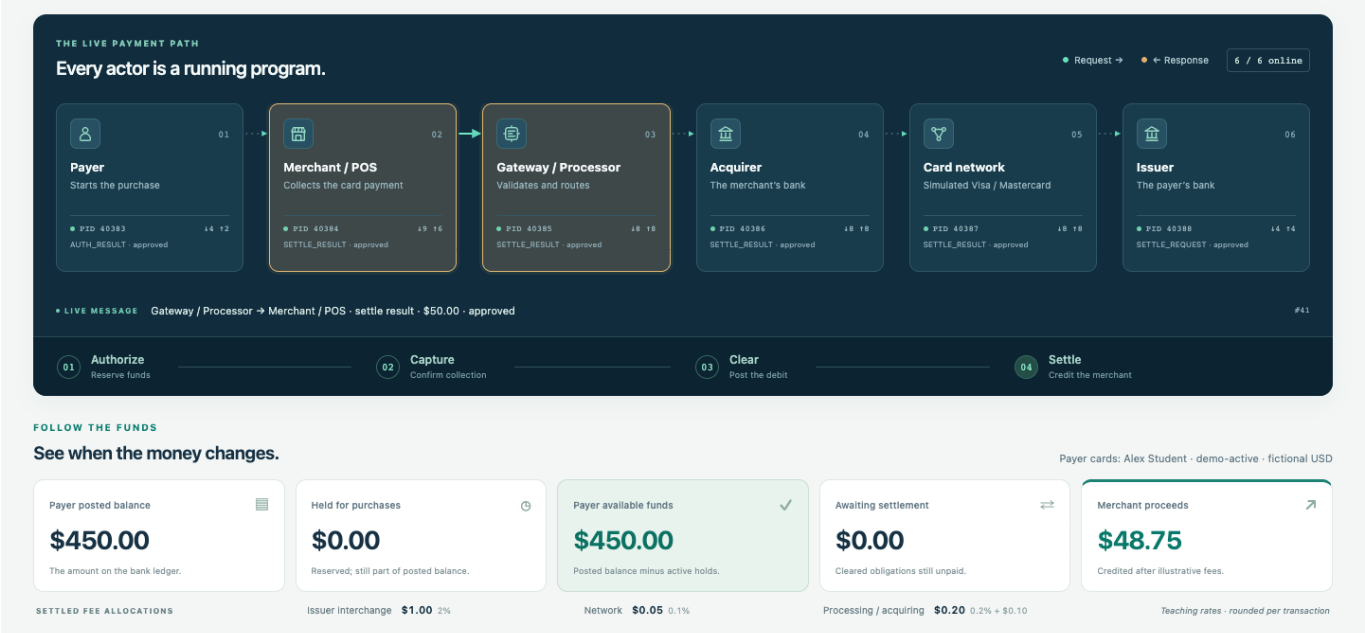
The reason should be visible along with the unchanged balances. A declined response is a completed issuer decision, not a missing message or a server fault.

The full captures also show the controls and message inspector. The financial panels above are participant state, not an animation-only illustration.

07 / INSTRUCTOR GUIDE

An eight-minute demonstration

Time	Action	Say / observe
0:00	Reset; point to six actors	Each role is a separate Python process. Find the payer’s \$500 opening balance.
0:45	Approved purchase; step	Follow who receives each message. The issuer is the account decision-maker.
2:00	Finish reverse response	Approval creates a hold. Posted balance is still \$500.
3:00	Capture; advance	The merchant is now requesting collection of the approved sale.
4:00	Clear Batch; advance	A posted debit replaces the hold; an unsettled obligation remains.
5:00	Settle Batch; advance	Merchant proceeds are \$48.75. Show the three fee components and the \$50 total.
6:15	Replay the operation	Inspect duplicate evidence. Explain why the same operation must not debit twice.
7:00	Reset; insufficient preset	Advance to the decline, then identify the reason and unchanged funds.



Settlement screenshot. The running app shows the settled payment and merchant/fee allocations. The diagram’s moving message is communication evidence; the ledger values are financial-state evidence.

Three audience prompts

1. At approval, which value changed: posted balance, held funds, or merchant proceeds?
2. After clearing, why did available funds not drop by another \$50?
3. Why does retrying the same operation produce evidence rather than another debit?

08 / INSTRUCTOR GUIDE

Inspect messages and exceptions

Select one transaction, then select a message in its event history. Read the plain-English explanation first. Expand the JSON to connect the explanation to the actual fields.

```
{
  "payment_id": "P0001",
  "operation_id": "P0001:authorize",
  "sender": "network", "recipient": "issuer",
  "type": "AUTHORIZE",
  "amount_cents": 5000, "currency": "USD",
  "outcome": "", "reason": ""
}
```

These are actual envelope field names; this abbreviated example omits the synthetic account, merchant, brand, and session metadata. Export the actual JSON trace to see exact values from your run. Preserve IDs across hops; sender and recipient change as the message travels.

Four useful comparison cases

Case	What to demonstrate
Insufficient funds	Valid active account; request exceeds available funds. Decline returns through the same path; no hold is added.
Inactive account	Account exists but cannot authorize. Compare the reason with insufficient funds.
Invalid account / input	Unknown synthetic token or invalid amount/currency is rejected visibly. Contract rejection can occur before the issuer.
Repeated operation	Replay preserves the operation identity. The recorded response is returned with no additional financial change.

Operations are ordered

Capture requires authorization. Clearing requires capture. Settlement requires a reconciled cleared batch. Pressing a later-stage control too early must expose a reason and leave funds unchanged. Wait until the queued response completes before narrating a stage as finished.

Automatic mode as a closing demonstration

Reset with the default seed. Start Automatic and adjust the speed. Watch both approvals and declines, with capture/clearing/settlement accelerated into seconds. Pause to inspect a payment. Use keyboard focus and shortcut hints; reduced motion preserves readable highlights while disabling travel animation. The random input sequence repeats with the same seed; process IDs and wall-clock timing do not.

09 / INSTRUCTOR GUIDE

Observation checklist and troubleshooting

Unscored instructor observation checklist

- ☐ Six different participant process IDs are visible and running.
- ☐ A request crosses all six actors, and an issuer result returns through every actor.
- ☐ Authorization adds a hold without lowering posted balance.
- ☐ Capture keeps the hold; clearing posts exactly one debit and releases it.
- ☐ Settlement removes the obligation and shows proceeds plus fees equal gross.
- ☐ Declines show a reason; replay leaves funds unchanged.
- ☐ A JSON trace matches the animated hop and displayed transaction.
- ☐ Reset clears the history and replaces all participant processes.

Resolve a classroom interruption

Symptom	Next action
Python command missing	Try <code>py -3</code> on Windows or install Python 3.9+. Check the README's prerequisite command.
Browser did not open	Keep the terminal running; manually open the printed local URL.
Address already in use	Stop an earlier run or launch with <code>-port 8011</code> .
Nothing moves in Guided	Press Next Message. Guided queues wait for instructor delivery.
Automatic has stopped	Check Pause, participant health, and terminal logs; select a slower speed to inspect.
Worker failure shown	Preserve the trace if useful, then Reset. The toy pauses rather than making up a response.
Unexpected state after refresh	Refresh observes the same supervisor session. Use Reset for fresh accounts.
Tiny projected text	Increase browser zoom and use a wide window. Read the selected message panel aloud.

What a failure means here

A crashed participant is different from a declined payment. A decline is a valid business result; a crash prevents the process from answering. The application marks the failure, pauses, and requires Reset. There is no promise of recovering financial state after a process or machine crash.

10 / INSTRUCTOR GUIDE

Artifacts, next steps, and sources

What is in Project 0

Artifact	Purpose
Runtime + browser assets	One-command, six-process demonstration with live events and balances.
Fictional fixtures + tests	Reproducible accounts/scenarios and financial/operation acceptance checks.
README + launchers	Quick start, controls, API examples, shutdown, and container instructions.
Course guide + LaTeX	This 10-page instructor walkthrough in PDF and editable source.
Technical design + LaTeX	Message/state rules, architecture, failure behavior, tests, and cloud evolution.
Mermaid + SVG diagrams	Architecture, authorization sequence, lifecycle, cloud evolution; editable and slide-ready.
Actual screenshots	Approval, decline, and settlement from the delivered running app.
AI-use + verification record	Generated-work provenance and checks that actually ran, with limitations.
Manifest + ZIP	File inventory and a portable full-source distribution.

A forward path without changing the lesson

Keep the local program for the first demonstration. Containerize the same six processes next. For a cloud classroom session, run the container on one Linux VM and open it through an SSH tunnel. Only later replace local queues with managed messaging and add durable state, authentication, and independently deployed participants. The supplement separates the working package from that future design.

Reading and provenance

- Adnan Masood, PhD., *Agentic Payments 101 (1/2): How the Card Payment System Works*, July 11, 2026. User-supplied article excerpt; conceptual teaching context.
- [VisaNet Connect – Acceptance: getting started](#). Primary documentation for authorization/capture and issuer-response routing.
- [Visa developer glossary](#). Definitions of issuer, acquirer, authorization, clearing, and settlement.
- [Stripe: Place a hold on a payment method](#). Primary documentation illustrating separate authorization and capture.

Sources checked October 6, 2026. Project 0's fee rates, queue transport, explicit stepping, and clearing-time debit are teaching choices. The simulated Visa/Mastercard box has no connection to either network.